Monocular ORB-Based Visual SLAM

Project report — Chinmayee Prabhakar

1. Objective

The objective was to estimate a moving camera's position and orientation from a sequence of monocular images while building a sparse 3D map. The project also aimed to make the estimates inspectable: a reader should be able to run the code, see accepted and missing poses, examine the reconstructed landmarks, and compare eligible trajectories with ground truth.

The implemented pipeline achieves short real-image tracking and sparse mapping. It does not achieve reliable continuous tracking across the full tested sequence. The distinction between an implemented component and a demonstrated benefit is kept throughout this report.

2. Introduction

A single camera records image coordinates, not direct depth. Matching a feature across different views makes it possible to estimate scene geometry and camera motion, but the reconstructed scale remains arbitrary without additional information. Incorrect matches, low camera displacement, occlusion, and changing viewpoints make the estimation difficult.

This is an independent Python implementation built around OpenCV, NumPy, and SciPy. OpenCV supplies ORB extraction, binary descriptor matching, and geometric solvers. Project-specific code manages initialization checks, camera states, landmark identities, keyframes, local mapping, optimization validation, and evaluation. It is not a complete implementation of the original ORB-SLAM system and does not claim equivalent performance.

This project was inspired by the material and public assignments from WPI's RBE/CS 549: Computer Vision course, taught by Prof. Nitin J. Sanket. I used those ideas to build an ORB-based system that tracks camera motion and builds a sparse 3D map, then tested it on KITTI. [1]

Evaluation uses KITTI odometry sequence 00, source frames 0–299 from the rectified left grayscale camera. Camera calibration is read from P0. Ground-truth poses are kept outside online estimation and used only after trajectories have been saved. The two 150-frame windows are fresh tracker restarts within the same data, not held-out sequences. [2]

Pipeline: image → ORB features → descriptor matches → two-view initialization → sparse landmarks → map-based PnP tracking → keyframes and map growth → validated local bundle adjustment.

On tracking failure, the camera pose remains missing. The current implementation starts a new map rather than joining uncertain trajectories. Each map has its own origin and monocular scale.

3. Methodology

3.1 Features and geometric initialization

ORB extracts up to 1,500 keypoints per image. Binary descriptors are matched using Hamming distance, a 0.75 nearest-neighbor ratio test, and a reverse-best consistency check. These filters reduce ambiguity but can discard useful matches when the available descriptors are not distinctive.

Initialization uses calibrated essential-matrix geometry, RANSAC, relative-pose recovery, and triangulation. Cheirality, parallax, reprojection quality, and deterministic match-order stability checks protect against weak or inconsistent two-view geometry. The stability audit requires enough fully accepted trials, rather than counting provisional rotations from rejected solutions. Conservative rejection prevents unsupported map creation at the cost of longer initialization gaps.

3.2 Persistent mapping and camera tracking

Landmarks retain stable identities and reciprocal image-observation records. New frames are matched to existing 3D points and localized using PnP-RANSAC. The minimum is 12 unique correspondences, then at least 12 RANSAC inliers and a 0.5 inlier fraction. Refined candidates also face finite-pose, positive-depth, and reprojection checks. A returned solver pose is not automatically an accepted online pose.

The pose convention is world-to-camera: $X_c = R_{cw} X_w + t_{cw}$. The camera center in the local world is $C_w = -R_{cw}^T t_{cw}$. This distinction is tested because confusing translation with camera center produces incorrect trajectories and visualizations.

Selected frames become keyframes. Local mapping triangulates additional landmarks, maintains bounded descriptor evidence, and enforces map-point capacity. Descriptor representatives use actual Hamming-distance medoids rather than averaging binary descriptors. Tracking failure isolates the old map from later independently initialized segments.

3.3 Local bundle adjustment and validation

Local BA uses sparse nonlinear least squares with a robust soft-L1 loss. Both initialization poses remain fixed to preserve the map's gauge and baseline. The solver budget is deliberately bounded. A candidate is applied only after validating training observations and all stored observations affected outside the optimization window. Rejected candidates must leave poses, points, descriptors, and observations unchanged.

An audited objective defect added a positive depth penalty to signed pixel residuals, allowing cancellation. The corrected per-observation residual is $[e_u, e_v, 100 * \sqrt{2} * \max(0, 0.0001 - z)]$. Its independent depth component cannot cancel image error. The sparse Jacobian has three rows per observation. This fixes the mathematics; it does not guarantee improved real-data tracking.

3.4 Offline evaluation

Accepted poses come only from per_frame.jsonl. Each eligible segment receives one positive-scale, nonreflecting Sim(3) alignment to KITTI. At least five poses and sufficient noncollinear motion are required; genuinely planar motion is allowed. Missing or ineligible values remain unavailable. Relative rotation and translation direction use exact original-frame gaps of 1, 5, and 10. Metric translation and scale diagnostics use one fixed segment alignment. A separate prefix-fit/suffix-score experiment checks later poses without refitting.

4. Results

4.1 Tracking and trajectory accuracy

Run	Accepted / requested	Missing	Maps	Longest span	Own-eligible ATE RMSE
A: 0–299	145/300 (48.33%)	155	9	34	0.184586 m; 58 poses
B: 0–149	76/150 (50.67%)	74	5	34	0.225032 m; 34 poses
C: 150–299	68/150 (45.33%)	82	3	33	Unavailable for full segments

These are in-sample, segment-wise aligned errors, not accuracy measurements for all accepted poses. Run C's full segments fail the declared conditioning rule; this does not mean every relative-motion measurement is unavailable. Scale-free relative errors remain measurable.

For segment 004, frames 88–121, fitting all 34 poses gives 0.225032 m RMSE. Fitting the first 17 and scoring the later 17 gives 0.614886 m. The increase illustrates the limits of interpreting a close in-sample trajectory fit as stable future accuracy. Run A's two eligible segment scales are 2.498513 and 8.195958 m per estimated unit. Their difference is expected to be unconstrained across independent maps and is not, by itself, evidence of within-map scale drift.

4.2 Component contributions and failure analysis

The corrected four-way ablation produced 97, 126, 145, and 145 accepted poses for fixed-landmark tracking, map growth, descriptor maintenance, and full BA respectively. This supports a benefit from map growth and descriptor updates on this fixed sequence. The ten full-run BA candidates all failed outside-window validation, so BA provided no published trajectory or map benefit.

The eight losses have two immediate mechanisms. At frames 43 and 207, only six and ten correspondences remained, below the PnP entry requirement. At frames 82 and 282, 40/83 and 31/70 inliers failed the fraction gate. Other losses failed absolute inlier support. Association and recovery audits did not establish enough independent identity evidence to adopt a new matcher or retained-map recovery policy.

4.3 Runtime and visual evidence

Run A took 26.116 s wall time. Recorded frame-processing median, p95, and maximum were 16.923, 202.858, and 2315.133 ms. Peak whole-process working set was 218.46 MiB. The reference host was an AMD Ryzen 7 4800H with 16 logical processors. These are observations from a specific machine, not guaranteed speed or SLAM-only memory. Large timing tails preclude a sustained real-time claim.

The demonstration contains two separate maps: segment 004 has 34 accepted poses and 670 final landmarks; segment 007 has 30 poses and 558 landmarks. Videos include the actual loss frames 122 and 282. The displayed maps are final offline point clouds, not progressive historical map snapshots. The reviewed implementation's complete local test log reported 156 passing tests before release-helper checks.

4.4 Design choices, challenges, and remaining limits

Choice or challenge	Response and trade-off
Ambiguous initialization	Stronger accepted-trial and geometry checks reduce false maps but reject more frames.
Preserving landmark identity	Persistent point IDs, reciprocal observations, and segment isolation make state changes testable.
BA fits its own window but harms other views	Outside-window validation and transactional rollback protect the accepted map; all tested KITTI candidates are rejected.
Depth penalty cancels pixel error	Separate the depth residual and update Jacobian structure; mathematical correctness improves without a measured trajectory gain.
Short tracking spans and long recovery gaps	Audit support, identities, scheduling, and bounded retained-map probes; no safe production correction is established.
Misleading visual or accuracy claims	Preserve missing poses, separate maps/scales, disclose scoring populations, and label offline alignment/final maps.

The main unresolved problem is reliable landmark reidentification over time. Sparse maps contain scatter and are not dense environmental reconstructions. There is no loop closure, validated relocalization, global map merging, or demonstrated long-term metric scale. Internal geometric acceptance alone does not certify every pose or landmark identity.

4.5 Reproduction and conclusion

README.md gives the exact data layout and commands. sequence_demo.py runs the core tracker. m5b_runner.py produces the three windows and repeats with Windows process profiling. scripts/evaluate_release.py reuses the existing offline scoring functions without requiring private historical milestone artifacts. m5b_figures.py plots the results; m5c_data.py, m5c_render.py, and m5c_offline_gt.py produce the two demonstration clips and the separate eligible GT view.

The repository keeps raw KITTI data and generated experiment directories out of Git. A fresh-checkout reproduction requires the original source images, calibration, and sensor timestamps. The public entry point explicitly reports historical comparisons as not performed. The original audit remains available when its historical artifacts are supplied; no missing historical audit is silently marked as passed.

The project demonstrates an end-to-end, testable monocular localization and sparse-mapping prototype. Its strongest result is short real-image tracking with explicit geometric checks, reproducible evaluation, and visible failures. Robust continuous SLAM remains a future engineering goal rather than an achieved claim.

Sources

[1] Nitin J. Sanket, WPI RBE/CS 549: Computer Vision, public course material: https://rbe549.github.io/.

[2] Geiger, Lenz, and Urtasun, KITTI Vision Benchmark Suite: https://www.cvlibs.net/datasets/kitti/. Dataset and image-derived media retain the applicable CC BY-NC-SA 3.0 attribution.

[3] Project evidence: accepted M5-B REPORT.md and run summaries; M5-C REPORT.md and media manifest; M4.5-C BA correction report. The README describes how to regenerate the corresponding outputs.